

ESCUELA COLOMBIANA DE INGENIERÍA DESARROLLO ORIENTADO POR OBJETOS

Diseño y Pruebas.

2026-2 — Laboratorio 2/6

OBJETIVO

Evidenciar el logro de los siguientes resultados de aprendizaje:

1. Desarrollar una aplicación aplicando Desarrollo Orientado al Comportamiento (BDD) y Desarrollo Dirigido por Modelos (MDD).
2. Realizar diseños (ingeniería directa e inversa) utilizando una herramienta de modelado (astah)
3. Manejar pruebas de unidad usando un framework ([junit](#))
4. Apropiar nuevas clases consultando sus especificaciones ([API java](#))
5. Experimentar las prácticas XP :
Designing. Use [CRC](#) cards for design sessions. **Testing.** All code must have [unit tests](#).

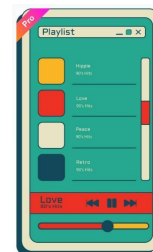
ENTREGA

- Incluyan en un archivo [.zip](#) los archivos correspondientes al laboratorio. El nombre debe ser los apellidos de los dos miembros del equipo ordenados alfabéticamente y separados por un guion. (CasallasC-DuarteJ)
- Publiquen el avance al final de la sesión y la versión definitiva en la fecha indicada, en los espacios correspondientes.

CONTEXTO

Objetivo: Desarrollar una versión simplificada de Tunes: [miniTunes](#)

[iTunes](#) es una aplicación para administrar colecciones musicales basada en estructuras como canciones, álbumes y listas de reproducción, que permiten organizar y manipular información musical de forma eficiente. En este laboratorio desarrollaremos una versión simplificada de [iTunes](#) utilizando únicamente listas de reproducción. Una lista de reproducción es una colección ordenada de canciones sobre la cual pueden realizarse operaciones de consulta, selección y combinación. En [miniTunes](#) cada canción estará caracterizada por su título, artista, género, duración y calificación. [iTunes/Music Tutorial](#)



PARTE I. Conociendo el proyecto

A Revisando línea base [En [lab02.pdf](#)]

1. El proyecto “miniTunes” contiene una construcción parcial del sistema. Revisen el directorio donde se encuentra el proyecto. Describan el contenido en términos de (a) directorios y de (b) las extensiones de los archivos.

El proyecto miniTunes se divide en un directorio de nombre miniTunes que contiene archivos de tipo .class, .ctxt, .java, dentro de ese directorio, existe otro directorio llamada doc que adentro contiene archivos .html, .jss, .css y .txt y dentro de este directorio existe otro directorio llamado resources que contiene un archivo .gif.

2. Exploren el proyecto en BlueJ

(a) ¿Cuántas clases tiene?

El proyecto cuenta con 3 clases

(b) ¿Cuál es la relación entre ellas?

La aplicación MiniTunes tiene a Playlist y para que verificar que estas playlist funcionan correctamente se le deben aplicar algunas pruebas de la clase PlaylistTest

(c) ¿Cuál es la clase principal de la aplicación?

La clase principal de la aplicación es MiniTunes.

(d) ¿Cómo la reconocen?

La reconozco porque es la que ofrece el servicio.

(e) ¿Cuál es la clase “diferente”?

La clase diferente es PlaylistTest.

(f) ¿Cuál es su propósito?

El propósito de esta clase es contener pruebas unitarias que pueden ser ejecutadas sobre instancias de la clase Playlist.

Para las siguientes dos preguntas sólo consideren las clases “normales”:

1. Generen y revisen la documentación del proyecto: ¿está completa la documentación de cada clase? (Detallen el estado de documentación: encabezado y métodos)

La documentación está incompleta ya que en los métodos falta detallar que representan las variables de los parámetros y que retorna, además los métodos no tienen cuerpo.

2. Revisen el código fuente del proyecto, ¿en qué estado está cada clase? (Detallen el estado de las fuentes considerando dos dimensiones: (a) la primera, atributos y métodos, y la segunda, (b) código, documentación y comentarios)

Clase MiniTunes

Esta contiene un atributo y 9 métodos los cuales contienen los parámetros que deben recibir pero no tienen nada de código en el cuerpo, en tanto a documentación y comentarios, estos contienen la explicación tipo caja negra de lo que hace el método.

Clase Playlist

Esta no contiene atributos y 8 métodos con nombre y los atributos que reciben pero sin una explicación de lo que hacen, en tanto a documentación y comentarios, contiene comentarios de el formato y atributos que debe contener cada canción que vaya a ser agregada en una playlist, dando algunas especificaciones de la robustez con la que debe contar el código para no fallar.

(c) ¿Qué diferencia hay entre el código, la documentación y los comentarios?

El código es la implementación del diseño de un programa en un lenguaje de programación, este se puede ejecutar.

La documentación es la explicación de que hace cada clase o método, principalmente es para que un usuario pueda guiarse en cómo funciona el programa sin necesidad de entender el código.

Los comentarios son una explicación de cómo funciona el programa internamente para quien lo programa.

B Realizando ingeniería inversa [En [lab02.pdf](#) [miniTunes.astah](#)]

MDD: MODEL DRIVEN DEVELOPMENT

1. Completen el diagrama de clases correspondiente al proyecto. (No incluyan la clase de pruebas)

2. (a) ¿Cuáles contenedores están definidos?

Los contenedores definidos son: `TreeMap<String, Playlist>`, `String[]`, `String[][]`

(b) ¿Qué diferencias hay entre el nuevo contenedor y el `ArrayList` y el `vector []` que conocemos?
Consulte el API de java.

Las diferencias entre `TreeMap` y `ArrayList` son las siguientes:

- `TreeMap` asigna pares de clave - valor, `ArrayList` y el `vector[]` asignan elementos.
- El acceso a `TreeMap` es por medio de una clave, para acceder a un `ArrayList` y un `vector[]` se hace por medio de índices.
- `TreeMap` y `ArrayList` tienen tamaño dinámico, `vector[]` tiene un tamaño fijo que se establece en su definición.
- `TreeMap` tiene las claves ordenadas, `ArrayList` mantiene orden de inserción y `vector[]` ordena por posiciones.
- `TreeMap` no permite claves repetidas, `ArrayList` y `vector[]` si lo permiten.
- El acceso es distinto:
 - `TreeMap` es: `map.get(clave)`
 - `Arraylist` es: `lista.get(índice)`
 - `vector[]` es: `vector[índice]`

3. En el nuevo contenedor,

(a) ¿Cómo adicionamos un elemento?

Suponiendo un `TreeMap <String, Playlist> playlists`, si se quiere agregar un par clave valor sería de la siguiente manera:
`playlists.put("Comfortably numb", "Pink Floyd");`

(b) ¿Cómo lo consultamos?

Para consultarlo sería de la siguiente manera:
`playlists.get("Comfortably numb");`
se consulta usando la clave.

(c) ¿Cómo lo eliminamos?

Para eliminarlo sería de la siguiente manera:
`playlist.remove("Comfortably numb");`
se elimina usando la clave.

PARTE II. Aprendiendo a probar

A Conociendo Pruebas en BlueJ [En lab02.pdf *.java]

BDD

Para poder cumplir con las prácticas XP vamos a aprender a realizar las pruebas de unidad usando las herramientas apropiadas. Para eso implementaremos algunos métodos en la clase Playlist.

1. Revisen el código de la clase PlaylistTest

(a) ¿cuáles etiquetas tiene (componentes con símbolo @)?

Tiene las etiquetas @Before, @Test, @After.

(b) ¿cuántos métodos tiene?

Tiene 6 métodos.

(c) ¿cuántos métodos son de prueba?

Tiene 4 métodos de prueba.

(d) ¿cómo los reconocen?

Se reconocen por que tienen la etiqueta Test.

2. Ejecuten los tests de la clase PlaylistTest. (click derecho sobre la clase, Test All)

(a) ¿cuántas pruebas se ejecutan?

Se ejecutan 4 pruebas.

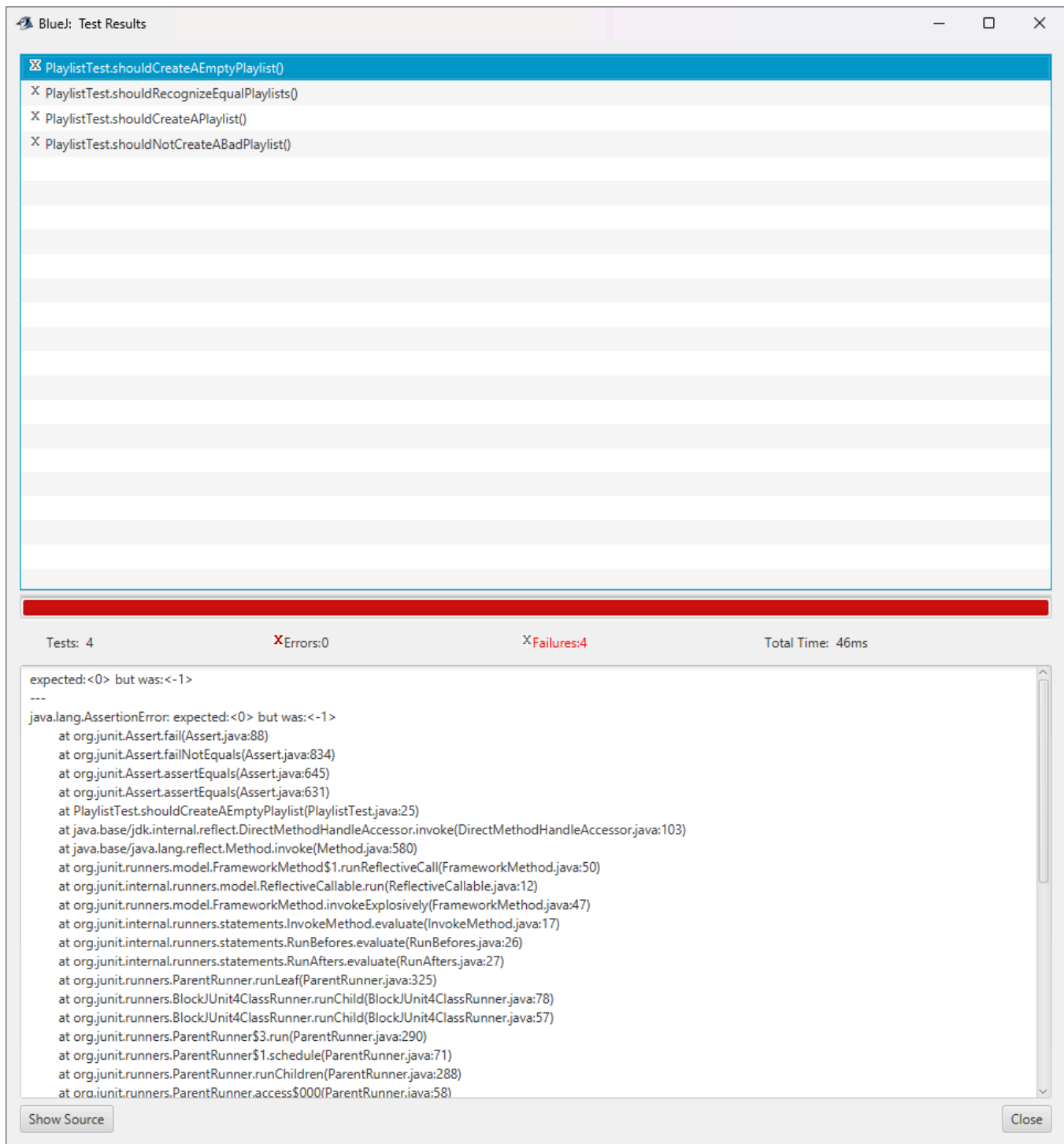
(b) ¿Cuántas pasan?

No pasa ninguna.

(c) ¿por qué?

Las cuatro pruebas fallan por AssertionError

(d) Capturen la pantalla.



3. Estudian las etiquetas encontradas en 1. Expliquen en sus palabras su significado.

@Test: Indica que es una prueba que JUnit debe de ejecutar.

@Before: Indica que el método con la etiqueta se ejecuta antes de cada método marcado con la etiqueta @Test.

@After: Indica que el método con la etiqueta se ejecuta después de cada método marcado con la etiqueta @Test.

4. Estudie los métodos `assertTrue`, `assertFalse`, `assertEquals`, `assertNull` y `fail` de la clase. `Assert` del API JUnit 1. Explique en sus palabras que hace cada uno de ellos.

Estos métodos pertenecen a la clase `Assert` y sirven para verificar si una condición se cumple a lo largo de una prueba.

`assertTrue`: Comprueba que una condición sea verdadera.

`assertFalse`: Comprueba que una condición sea falsa.

`assertEquals`: Comprueba que dos objetos o dos valores sean iguales.

`assertNull`: Comprueba que un valor sea `null`, el valor puede venir de un atributo, método o variable.

`fail()`: Fuerza una prueba a fallar.

5. Investiguen y expliquen la diferencia entre un fallo y un error en Junit. Escriba código, usando los métodos del punto 4., para codificar los siguientes tres casos de prueba y lograr que se comporten como lo prometen `shouldPass`, `shouldFail`, `shouldErr`.

En JUnit los fallos ocurren cuando un `assert` no se cumple, los errores ocurren cuando durante la ejecución del programa tuvo un problema durante la ejecución por ejemplo una división por cero o un `NullPointerException`.

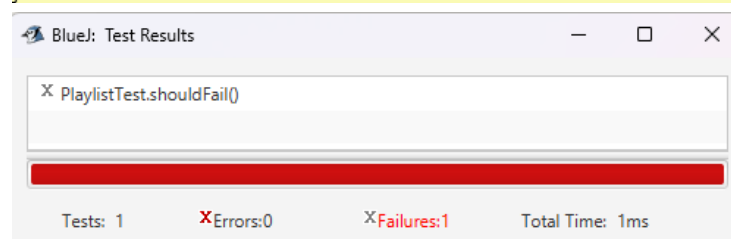
`shouldPass`:

```
@Test
public void shouldPass(){
    String [][] songs=
        {{"One", "U2", "Rock", "4", "*****"},
         {"Numb", "Linkin Park", "Rock", "Rock", null},
         {"Alive", "Pearl Jam", "Rock", "5", "****"},
         {"Creep", null, "Rock", null, "*****"},
         {null, "Fleetwood Mac", null, "4", "*****"}};
    assertEquals(new Playlist(songs),new Playlist(songs));
}
```

`shouldPass() succeeded`

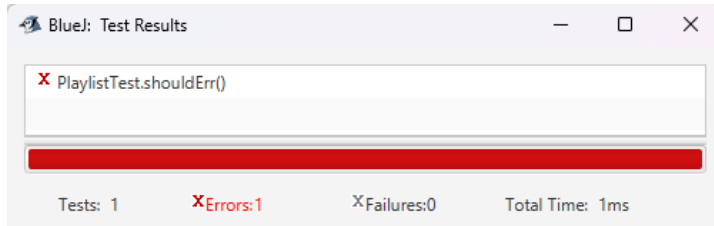
`shouldFail`:

```
@Test
public void shouldFail(){
    String [][] songs=
        {{"One", "U2", "Rock", "4", "*****"},
         {"Numb", "Linkin Park", "Rock", "Rock", null},
         {"Alive", "Pearl Jam", "Rock", "5", "****"},
         {"Creep", null, "Rock", null, "*****"},
         {null, "Fleetwood Mac", null, "4", "*****"}};
    assertEquals(new Playlist(songs),new Playlist(songs));
}
```



shouldErr:

```
@Test
public void shouldErr(){
    String [][] songs = {};
    Playlist pl = new Playlist(songs);
    String[] song = songs[0];
}
```



B Practicando Pruebas en BlueJ [En lab02.pdf *.java]

BDD

Ahora vamos a escribir el código necesario para que las pruebas de pasen `PlaylistTest`.

1. (a) **Determinen los atributos de la clase `Playlist`.**

```
private String[][] song;
```

- (b) **Justifiquen la selección.**

Solo es necesario este atributo porque guarda cada canción con toda la información que se necesita en la forma de una matriz.

2. (a) **Determinen el invariante de la clase `Playlist`.**

No existen dos canciones con la misma combinación de título de artista.

- (b) **Justifiquen la decisión.**

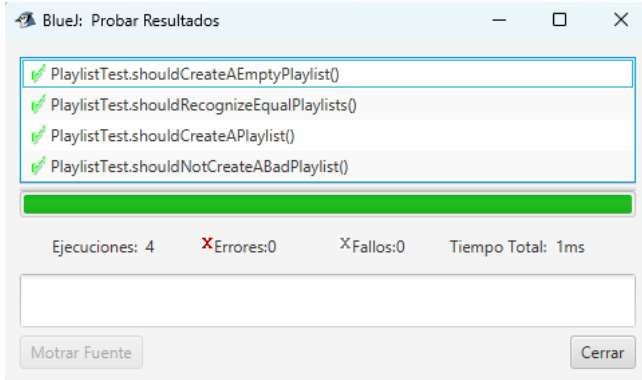
A nuestro parecer de las condiciones dadas en la documentación, esta es la más importante ya que no le vemos el sentido a que un artista tuviera dos canciones exactamente con el mismo título.

3. **Implementen los métodos de `Playlist` necesarios para pasar todas las pruebas definidas.**

- (a) **¿Cuáles métodos implementaron?**

Se implementaron los métodos
`public Playlist(String[][] songs)`
`public Playlist add(String[] song)`
`public int size()`
`public boolean equals(Playlist pl)`

4. Capturen los resultados de las pruebas de unidad.



PARTE III. Desarrollando el proyecto `miniTunes`. [En `lab02.pdf` `miniTunes.astah *.java`]

MDD-BDD

Para desarrollar esta aplicación vamos a considerar algunos ciclos.

En cada ciclo deben realizar los pasos definidos a continuación.

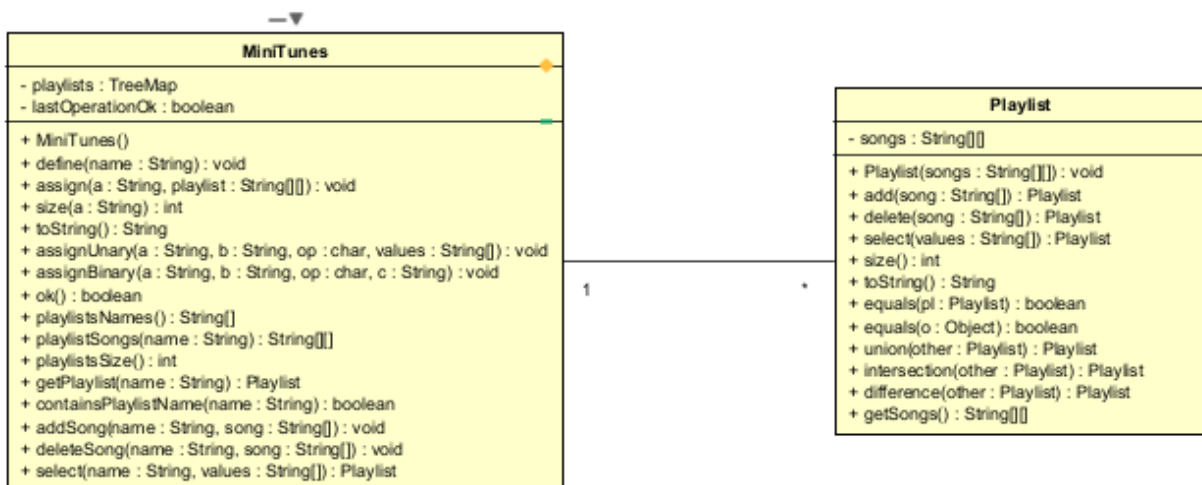
- I Definir los métodos base correspondientes al mini-ciclo actual.
- II Definir y programar los casos de prueba de esos métodos.
Piensen en los debería y los noDebería (`should...` y `shouldNot...`)
- III Diseñar los métodos
Usen diagramas de secuencia. En astah, crean el diagrama sobre el método correspondiente.
- IV Escribir el código correspondiente (no olvide la documentación)
- V Ejecutar las pruebas de unidad (vuelva a 3 (a veces a 2), si no están en verde)
- VI Completar la tabla de clases y métodos. (Al final del documento)

Ciclo 1 : Operaciones básicas: definir el nombre de una lista, asignar a un nombre una lista, consultar nombres de listas y consultar las canciones de una lista.

Ciclo 2: Operaciones unarias: adicionar una canción, eliminar una canción y seleccionar canciones dada una condición.

Ciclo 3 : Operaciones binarias: unión, intersección y diferencia.

BONO Ciclo 4 : Defina e implemente tres nuevas operaciones



Completen la siguiente tabla indicando el número de ciclo y los métodos asociados de cada clase.

Ciclo	MiniTunes	MiniTunesTest
-------	-----------	---------------

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)
Samuel Ahumada 12 horas.
Nerieth Sofia 12 horas.
2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?
Terminado
3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?
TDD ya que es bastante útil para identificar los casos límites de los métodos a crear.
4. ¿Cuál consideran fue el mayor logro? ¿Por qué?
Aprender a estructurar las pruebas unitarias como parte del diseño, ya que hace que el desarrollo de los métodos sea más modular.
5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
Los diagramas de secuencia en astah ya que varias cosas que necesitaban implementarse como los loops o condicionales no lo sabíamos hacer, así que tuvimos que buscar imágenes y videos en internet para aprender a hacerlo.
6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
Trabajar a la par y hacer un repaso general a todo el laboratorio para identificar vacíos de comprensión y resolverlos, nos comprometemos a organizarnos mejor para reunirnos y trabajar en los proyectos y laboratorios.
7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

JUnit 4 - Assert

<https://junit.org/junit4/javadoc/4.13/org/junit/Assert.html>

Java - String

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html>

Java - TreeMap

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/TreeMap.html>

Consideramos que la más útil es JUnit ya que en gran parte el propósito del laboratorio es Aprender a realizar pruebas unitarias con el uso de esta librería externa.